LAPORAN PRAKTIKUM

Pemrograman Website - Bab 7: Membangun REST API dengan Express.js
User Management REST API (Bacapedia+)

Nama : Steve Valentino Wijaya
NPM : 242310026
Kelas : TI-24-KA

1. Tujuan

- Membangun backend REST API menggunakan Node.js, Express.js, dan ORM Sequelize.
- Membuat model dan migration untuk table users.
- Mengimplementasikan endpoint CRUD user management (GET, POST, PUT, DELETE).
- Menguji seluruh endpoint menggunakan Postman.

2. Struktur Project Backend

backend/

- config/config.js -> konfigurasi koneksi database Sequelize
- models/user.js, models/book.js -> definisi model ORM
- migrations/ -> struktur table books & users
- controllers/userController.js -> logika CRUD + login (bcrypt, JWT)
- routes/userRoutes.js -> pemetaan endpoint /api/users
- middleware/auth.js -> verifikasi token JWT untuk protected routes

3. Endpoint REST API Users

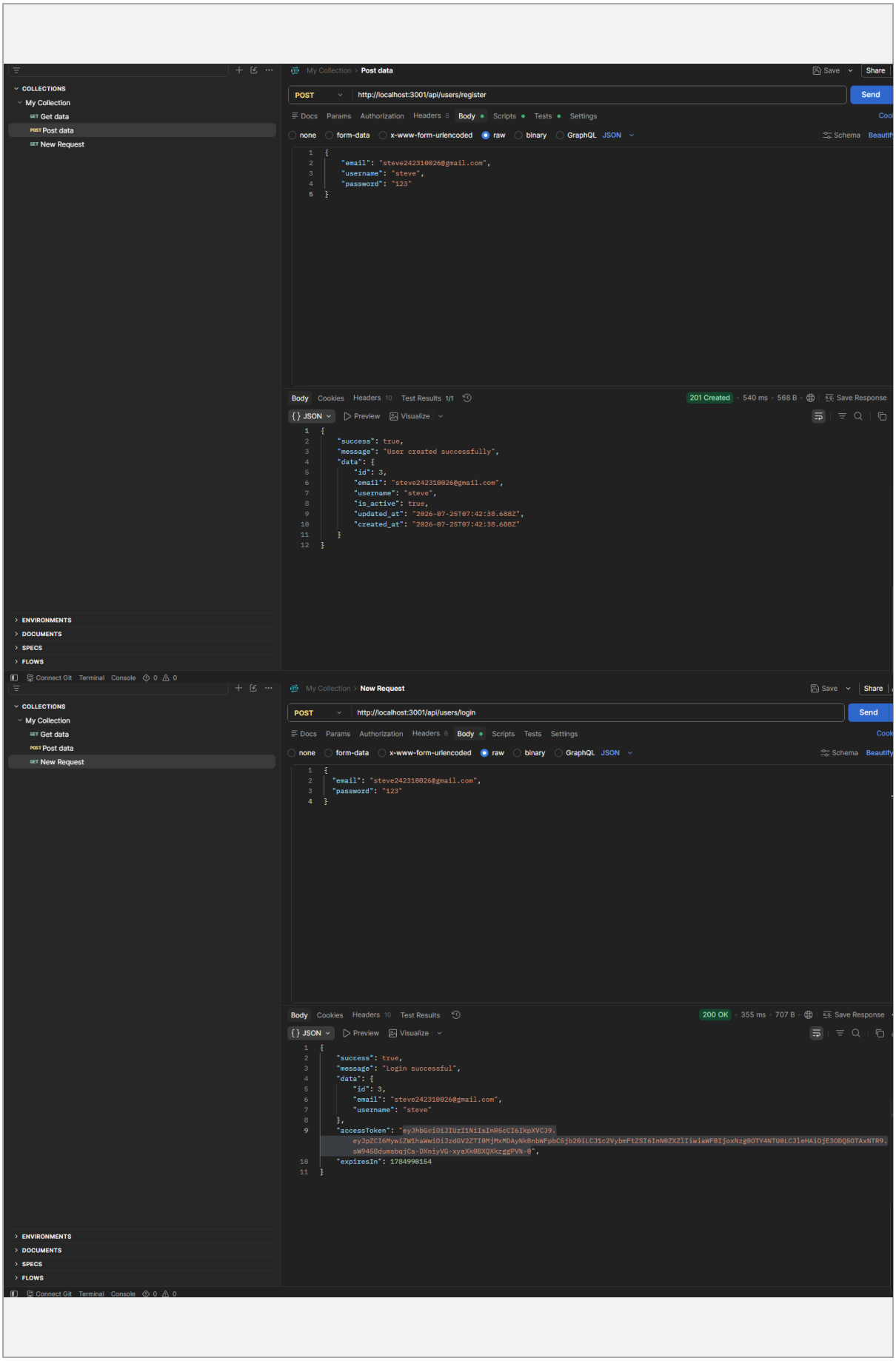
- GET /api/users -> mengambil seluruh data user
- GET /api/users/:id -> mengambil detail user berdasarkan ID
- POST /api/users/register -> menambah user baru (password di-hash bcrypt)
- POST /api/users/login -> autentikasi user, mengembalikan JWT token
- PUT /api/users/:id -> memperbarui data user (protected, perlu token)
- DELETE /api/users/:id -> menghapus user (protected, perlu token)

4. Perbaikan Bug yang Ditemukan

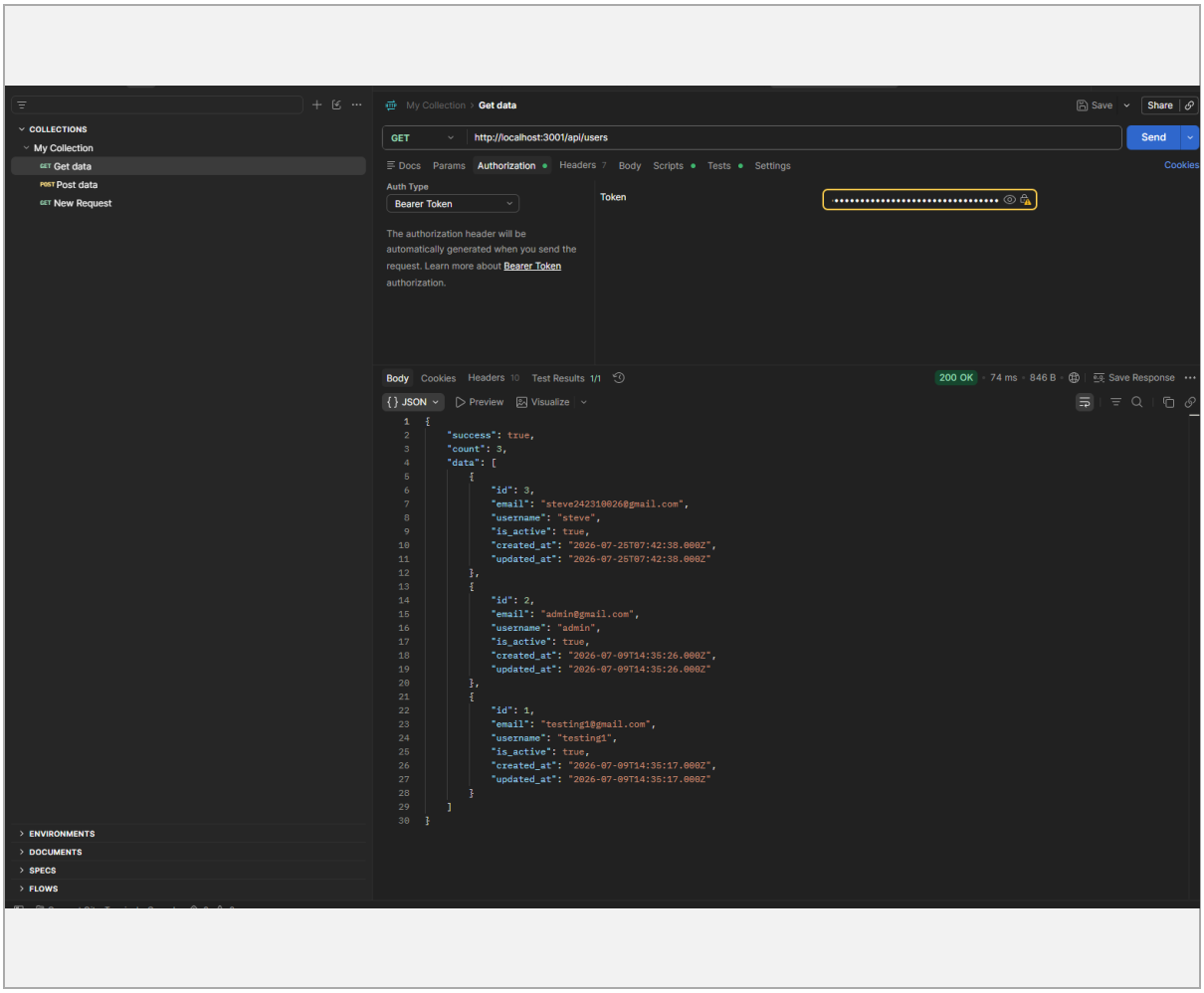
- Migration create-book.js dan create-user.js membuat tabel dengan nama "Books"/"Users" (huruf besar), sementara model mendefinisikan tableName "books"/"users" (huruf kecil) -> disamakan agar konsisten dan tidak error di database case-sensitive.
- package.json backend belum memiliki script "dev"/"start" sehingga npm run dev gagal -> ditambahkan script node index.js dan nodemon index.js.

5. Screenshot Pengujian API (Postman)

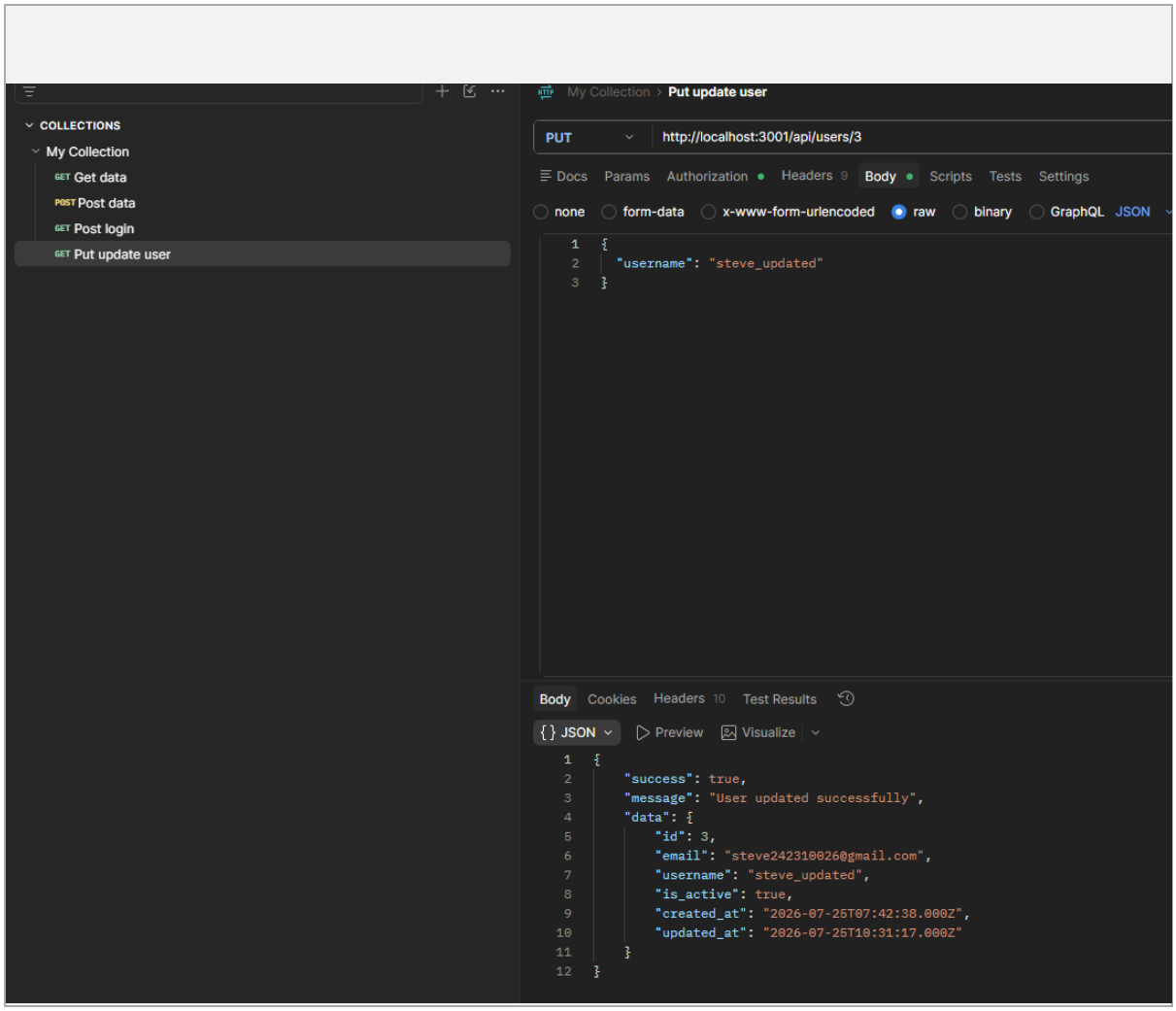
Login user (POST /api/users/login) - mendapatkan access token:

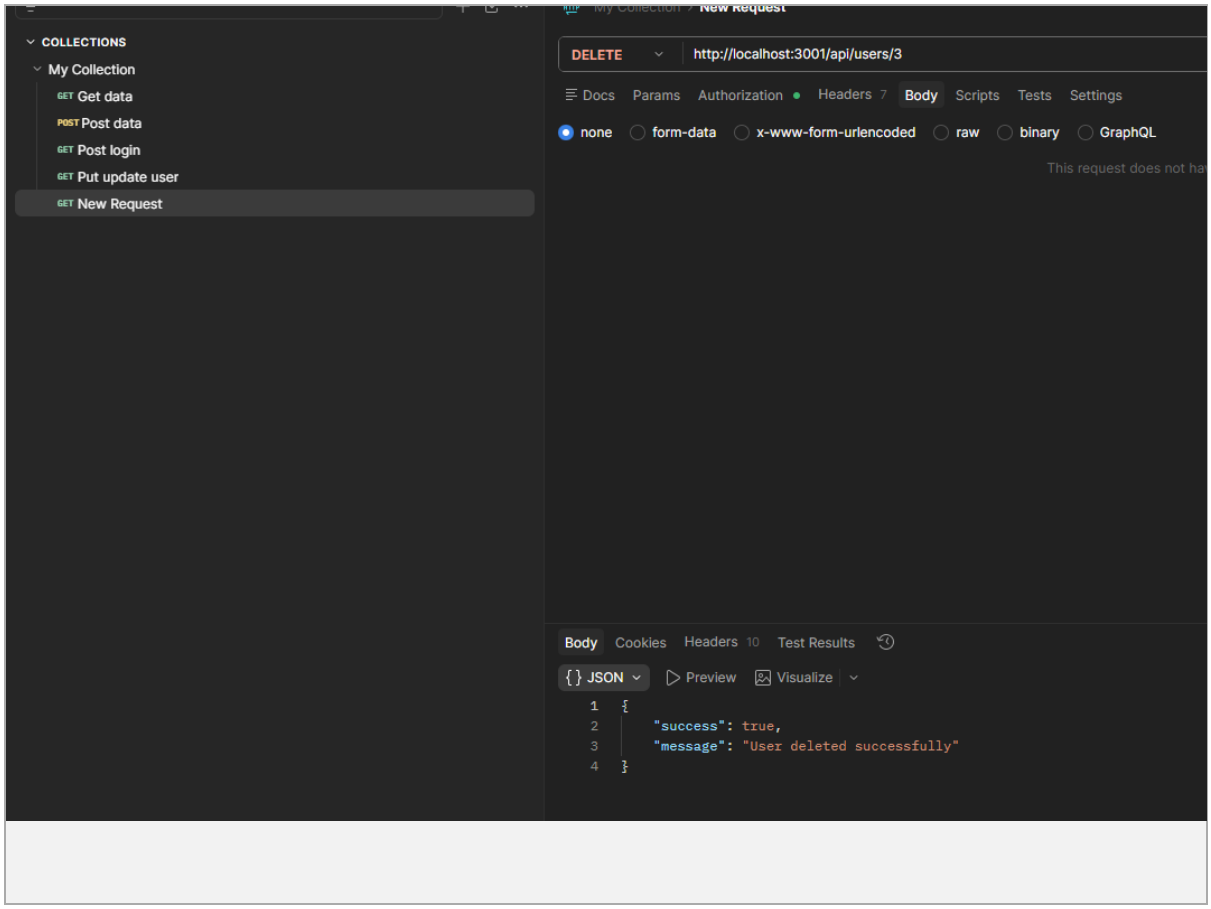


Get All Users (GET /api/users) dengan header Authorization Bearer token:



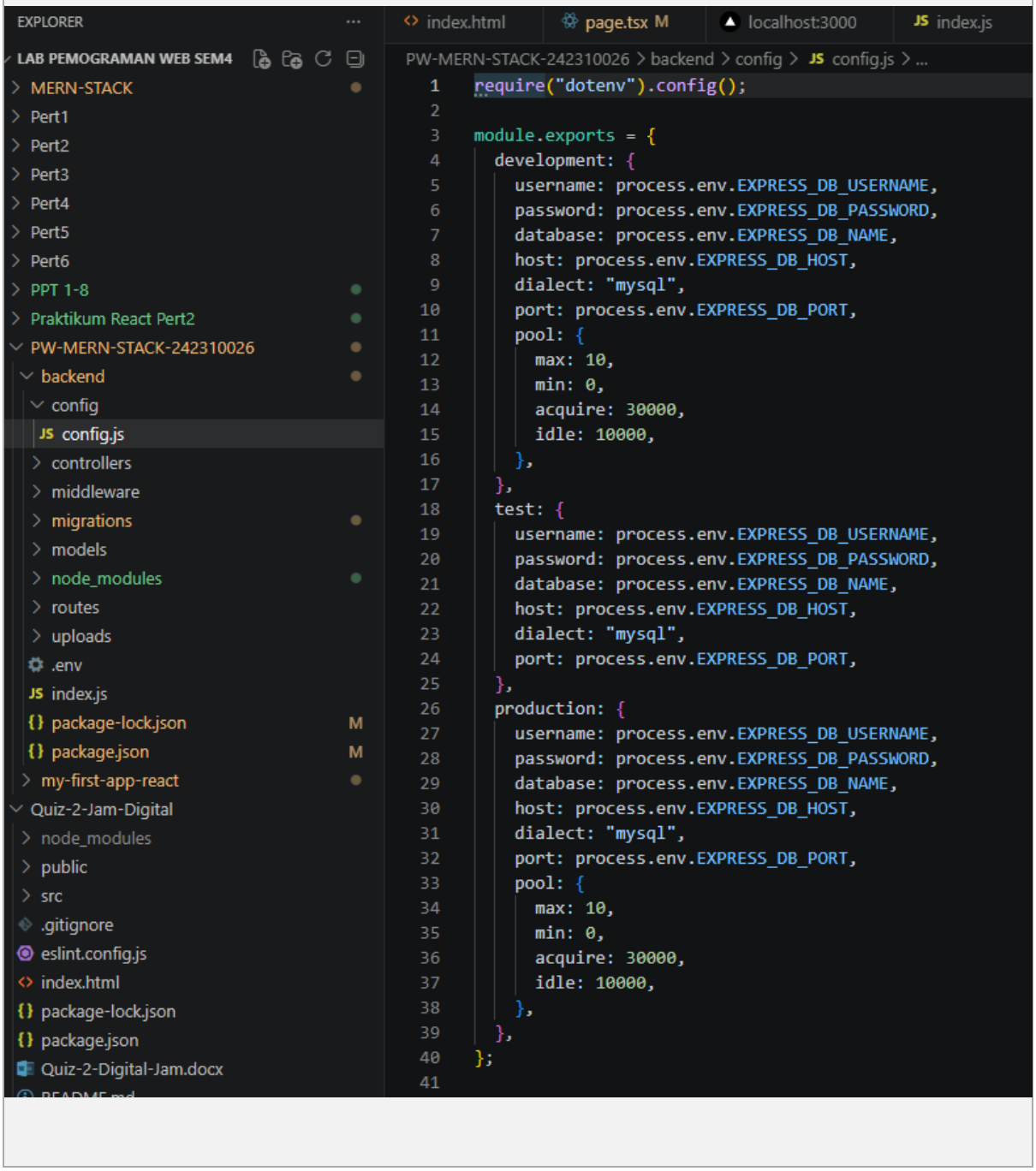
Create / Update / Delete User:


COLLECTIONS
My Collection
GET Get data
POST Post data
GET Post login
GET Put update user
My Collection > Put update user
PUT
http://localhost:3001/api/users/3
Docs Params Authorization Headers 9 Body Scripts Tests Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
{
"username": "steve_updated"
}
Body Cookies Headers 10 Test Results
{} JSON Preview Visualize
{
"success": true,
"message": "User updated successfully",
"data": {
"id": 3,
"email": "steve242310026@gmail.com",
"username": "steve_updated",
"is_active": true,
"created_at": "2026-07-25T07:42:38.000Z",
"updated_at": "2026-07-25T10:31:17.000Z"
}
}

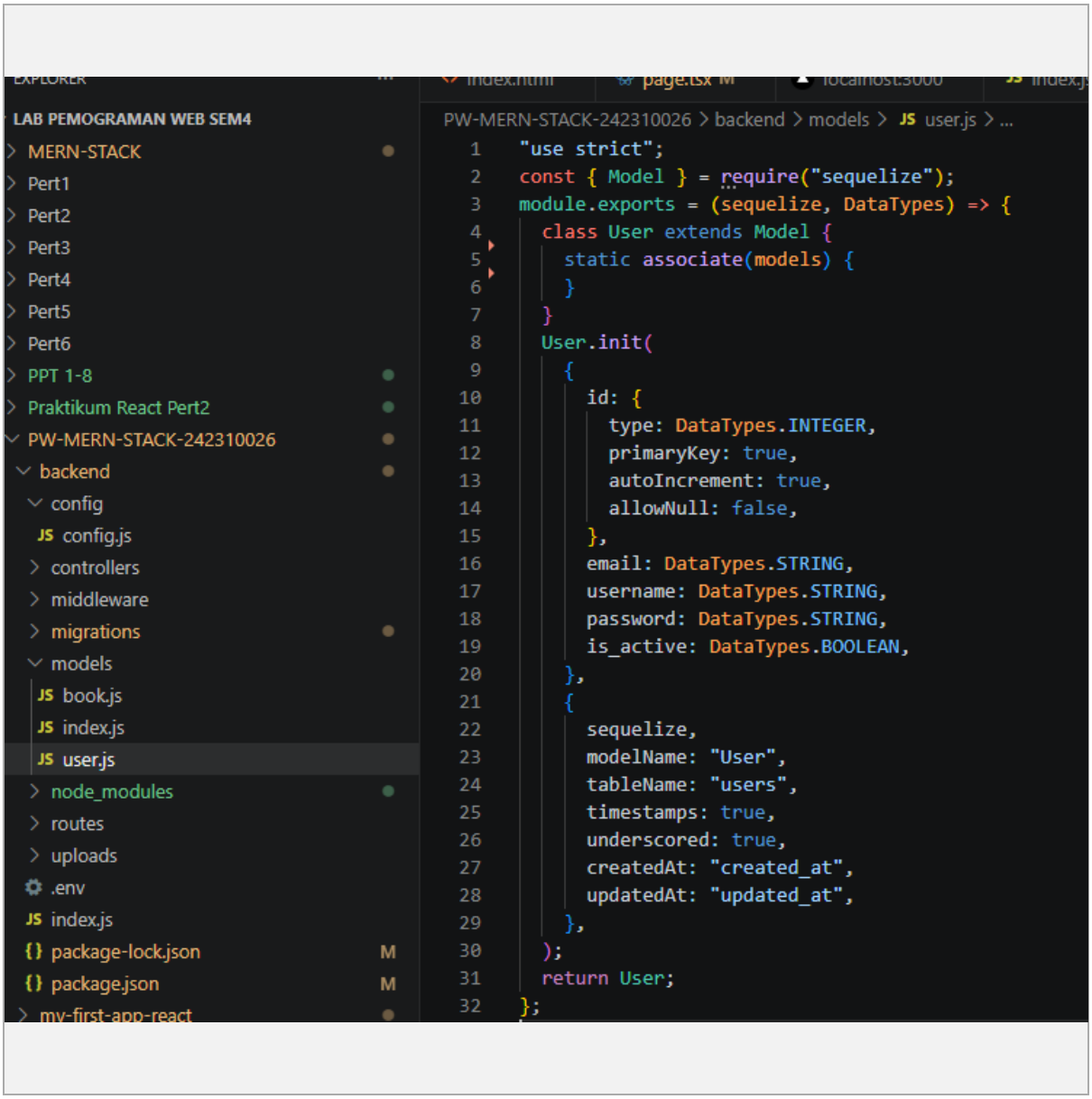


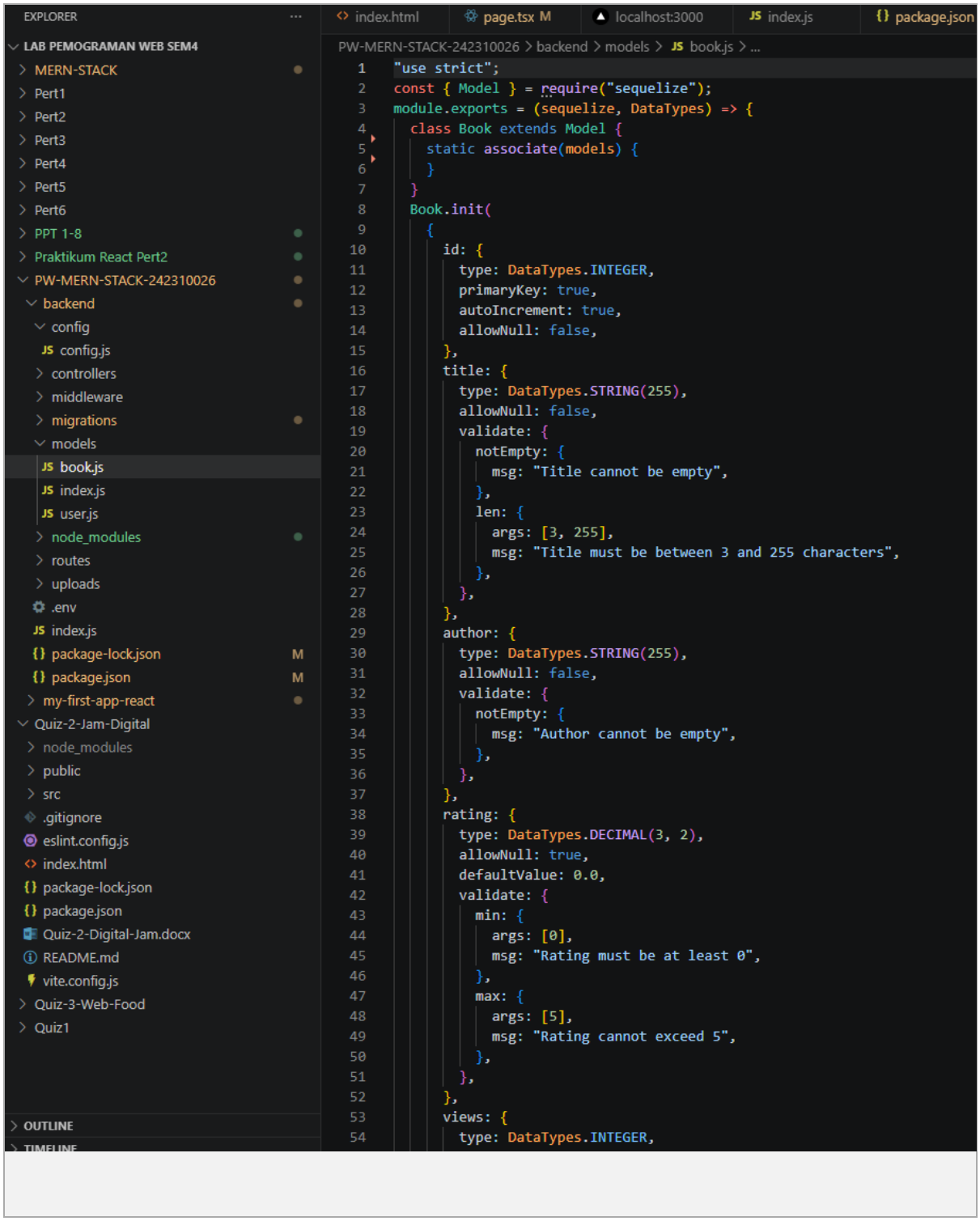
6. Screenshot Code yang Dimodifikasi

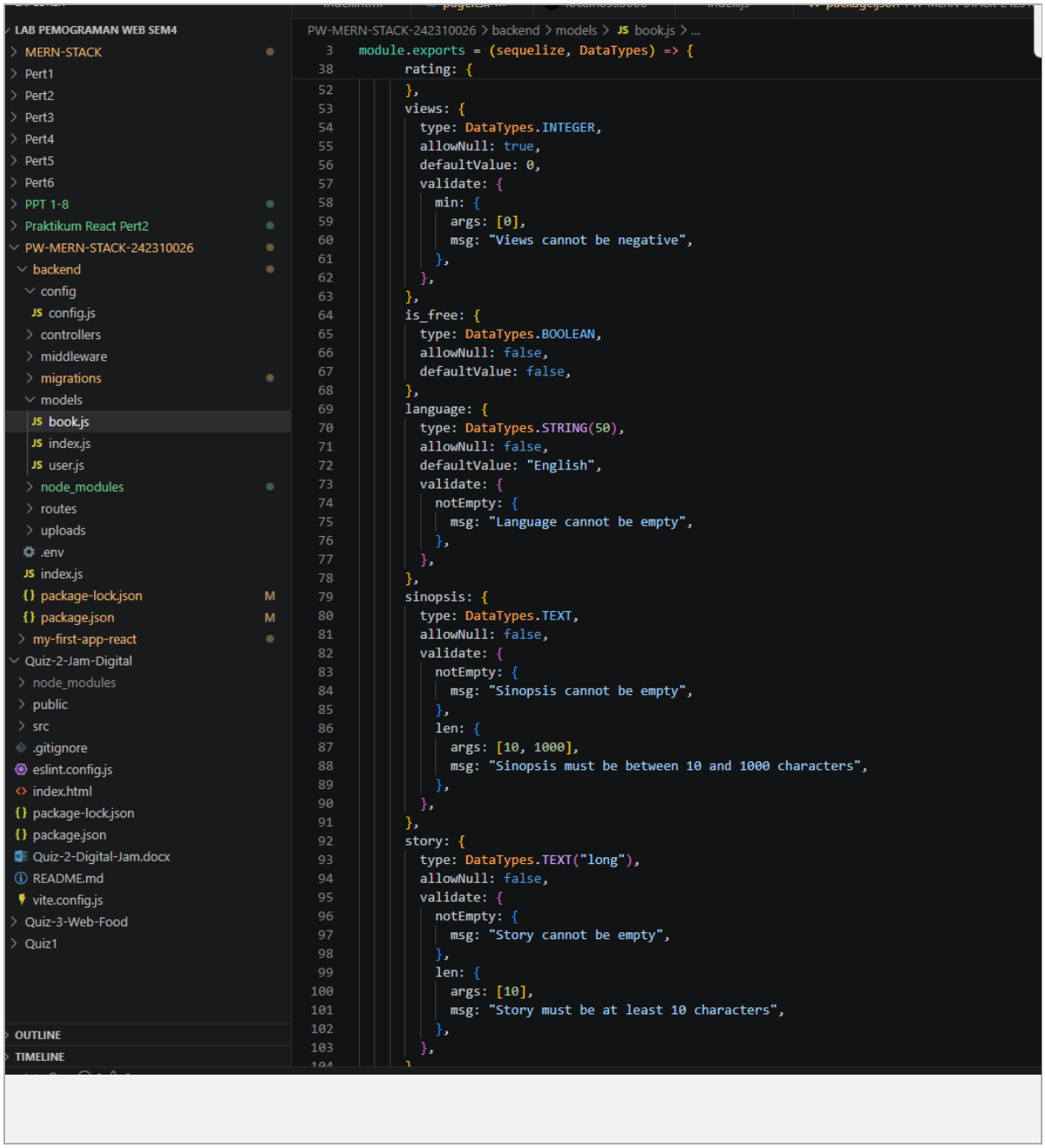
config/config.js -> konfigurasi koneksi database Sequelize

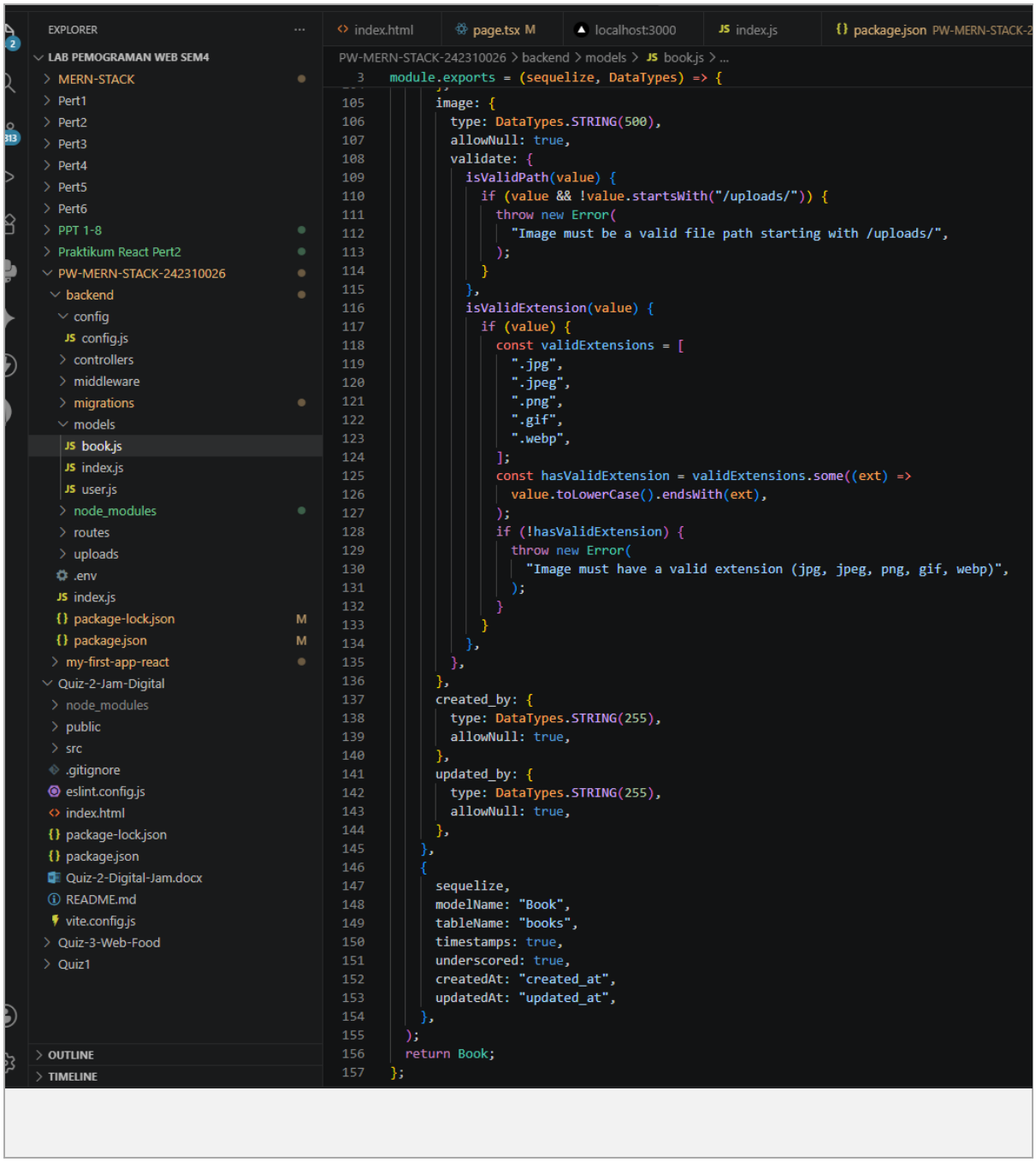

EXPLORER
LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
middleware
migrations
models
node_modules
routes
uploads
.env
index.js
package-lock.json M
package.json M
my-first-app-react
Quiz-2-Jam-Digital
node_modules
public
src
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
Quiz-2-Digital-Jam.docx
index.html
page.tsx M
localhost:3000
index.js
PW-MERN-STACK-242310026 > backend > config > config.js > ...
require("dotenv").config();
module.exports = {
development: {
username: process.env.EXPRESS_DB_USERNAME,
password: process.env.EXPRESS_DB_PASSWORD,
database: process.env.EXPRESS_DB_NAME,
host: process.env.EXPRESS_DB_HOST,
dialect: "mysql",
port: process.env.EXPRESS_DB_PORT,
pool: {
max: 10,
min: 0,
acquire: 30000,
idle: 10000,
},
},
test: {
username: process.env.EXPRESS_DB_USERNAME,
password: process.env.EXPRESS_DB_PASSWORD,
database: process.env.EXPRESS_DB_NAME,
host: process.env.EXPRESS_DB_HOST,
dialect: "mysql",
port: process.env.EXPRESS_DB_PORT,
},
production: {
username: process.env.EXPRESS_DB_USERNAME,
password: process.env.EXPRESS_DB_PASSWORD,
database: process.env.EXPRESS_DB_NAME,
host: process.env.EXPRESS_DB_HOST,
dialect: "mysql",
port: process.env.EXPRESS_DB_PORT,
pool: {
max: 10,
min: 0,
acquire: 30000,
idle: 10000,
},
},
};

models/user.js, models/book.js -> definisi model ORM

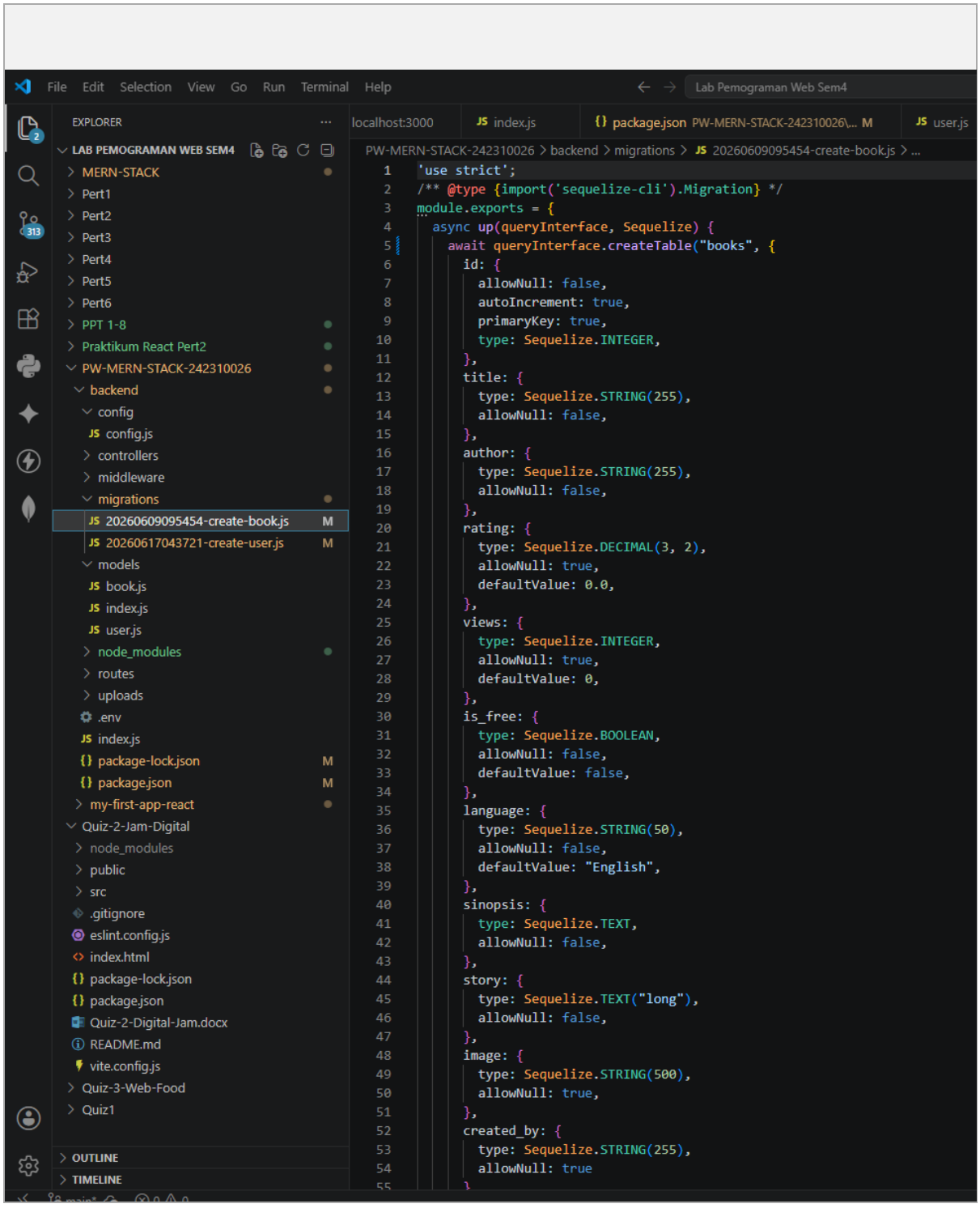


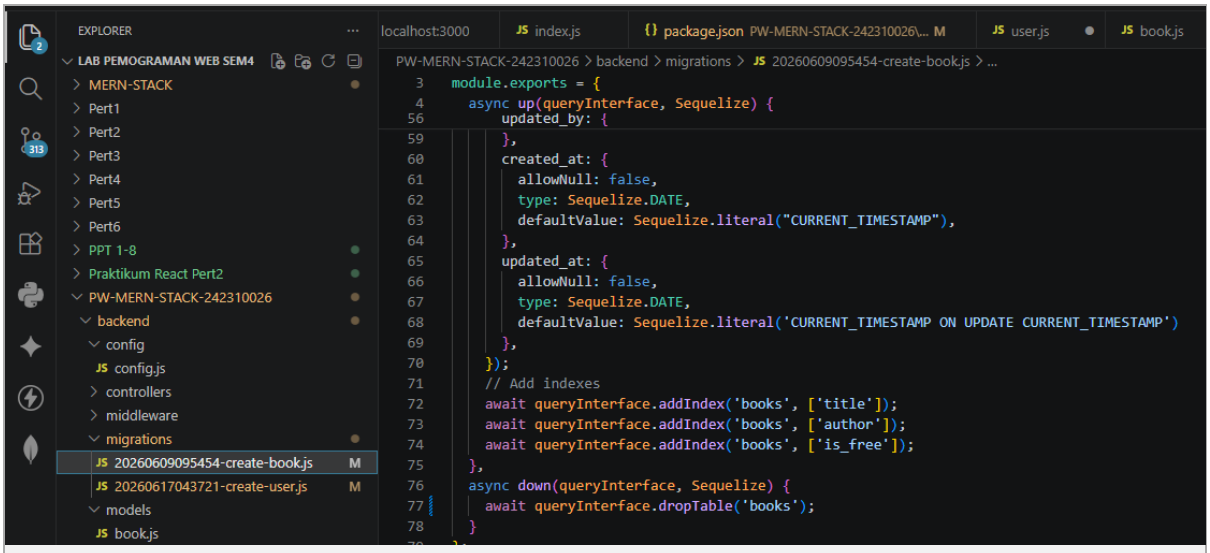

EXPLORER
LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
middleware
migrations
models
book.js
index.js
user.js
node_modules
routes
uploads
.env
index.js
package-lock.json
package.json
my-first-app-react
Quiz-2-Jam-Digital
node_modules
public
src
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
Quiz-2-Digital-Jam.docx
README.md
vite.config.js
Quiz-3-Web-Food
Quiz1
OUTLINE
TIMELINE
index.html
page.tsx M
localhost:3000
index.js
package.json
PW-MERN-STACK-242310026 > backend > models > book.js > ...
"use strict";
const { Model } = require("sequelize");
module.exports = (sequelize, DataTypes) => {
class Book extends Model {
static associate(models) {
}
}
Book.init(
{
id: {
type: DataTypes.INTEGER,
primaryKey: true,
autoIncrement: true,
allowNull: false,
},
title: {
type: DataTypes.STRING(255),
allowNull: false,
validate: {
notEmpty: {
msg: "Title cannot be empty",
},
len: {
args: [3, 255],
msg: "Title must be between 3 and 255 characters",
},
},
},
author: {
type: DataTypes.STRING(255),
allowNull: false,
validate: {
notEmpty: {
msg: "Author cannot be empty",
},
},
},
rating: {
type: DataTypes.DECIMAL(3, 2),
allowNull: true,
defaultValue: 0.0,
validate: {
min: {
args: [0],
msg: "Rating must be at least 0",
},
max: {
args: [5],
msg: "Rating cannot exceed 5",
},
},
},
views: {
type: DataTypes.INTEGER,

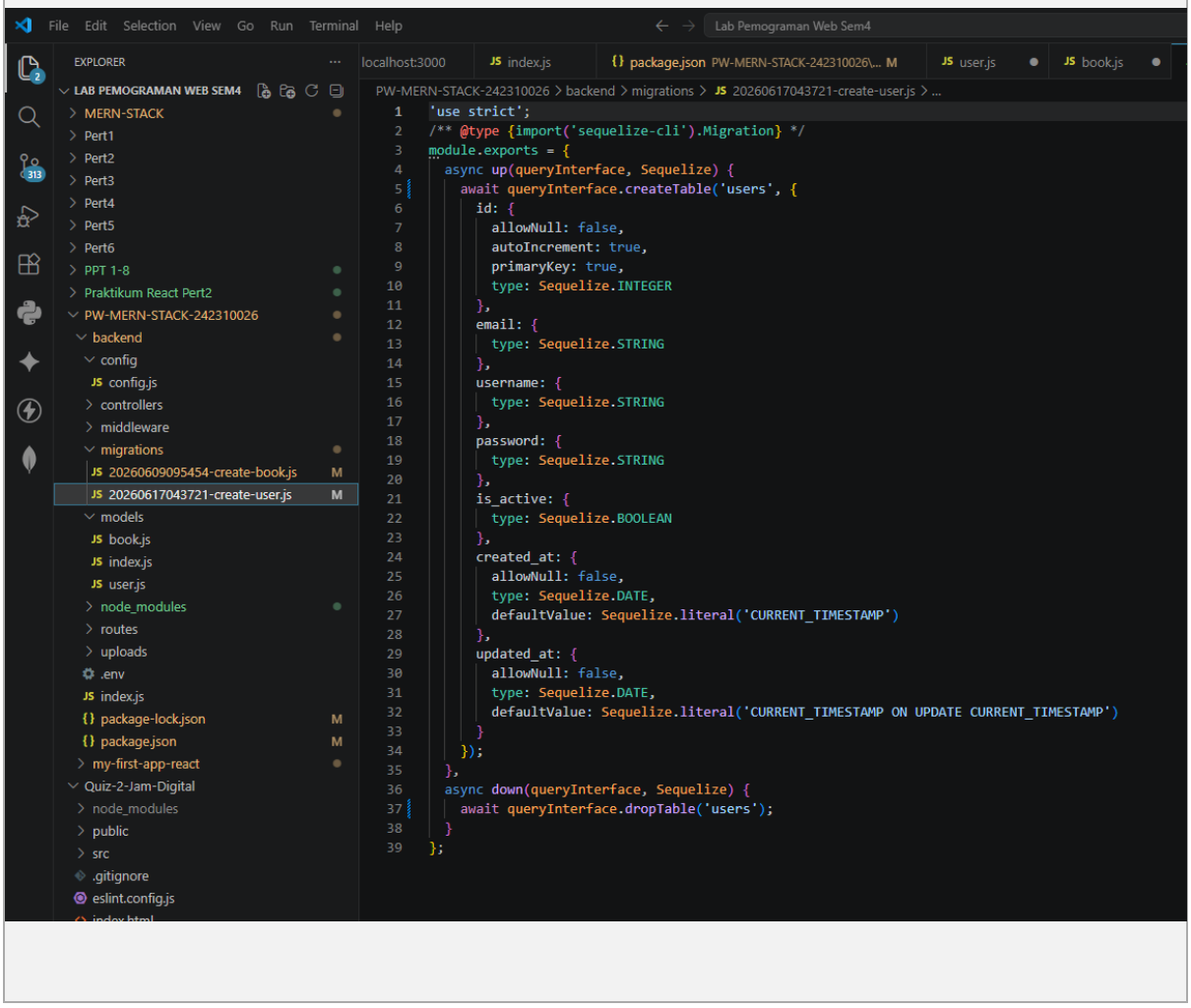

LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
middleware
migrations
models
book.js
index.js
user.js
node_modules
routes
uploads
.env
index.js
package-lock.json
package.json
my-first-app-react
Quiz-2-Jam-Digital
node_modules
public
src
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
Quiz-2-Digital-Jam.docx
README.md
vite.config.js
Quiz-3-Web-Food
Quiz1
OUTLINE
TIMELINE
PW-MERN-STACK-242310026 > backend > models > book.js > ...
3 module.exports = (sequelize, DataTypes) => {
38 rating: {
52 },
53 views: {
54 type: DataTypes.INTEGER,
55 allowNull: true,
56 defaultValue: 0,
57 validate: {
58 min: {
59 args: [0],
60 msg: "Views cannot be negative",
61 },
62 },
63 },
64 is_free: {
65 type: DataTypes.BOOLEAN,
66 allowNull: false,
67 defaultValue: false,
68 },
69 language: {
70 type: DataTypes.STRING(50),
71 allowNull: false,
72 defaultValue: "English",
73 validate: {
74 notEmpty: {
75 msg: "Language cannot be empty",
76 },
77 },
78 },
79 sinopsis: {
80 type: DataTypes.TEXT,
81 allowNull: false,
82 validate: {
83 notEmpty: {
84 msg: "Sinopsis cannot be empty",
85 },
86 len: {
87 args: [10, 1000],
88 msg: "Sinopsis must be between 10 and 1000 characters",
89 },
90 },
91 },
92 story: {
93 type: DataTypes.TEXT("long"),
94 allowNull: false,
95 validate: {
96 notEmpty: {
97 msg: "Story cannot be empty",
98 },
99 len: {
100 args: [10],
101 msg: "Story must be at least 10 characters",
102 },
103 },



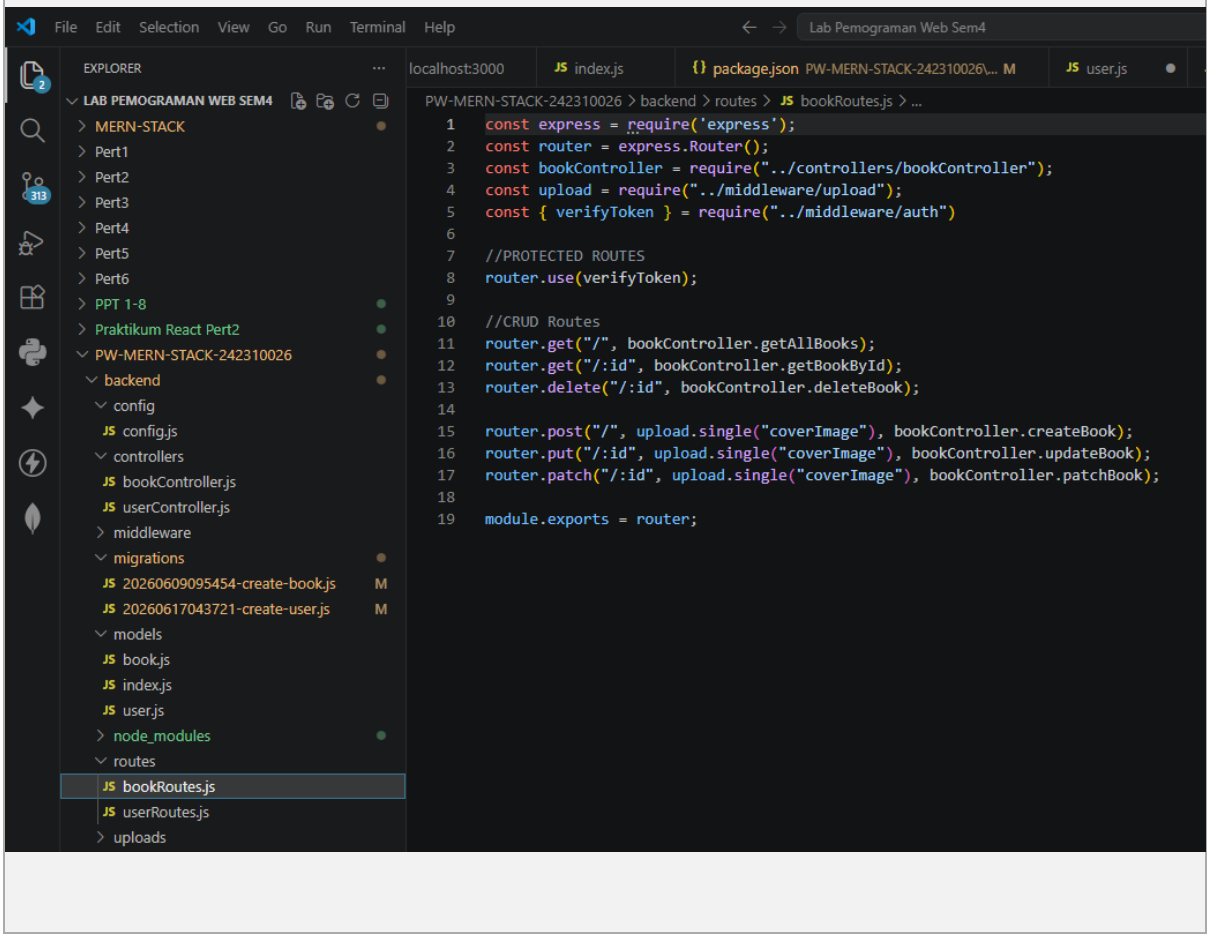
migrations/ -> struktur table books & users

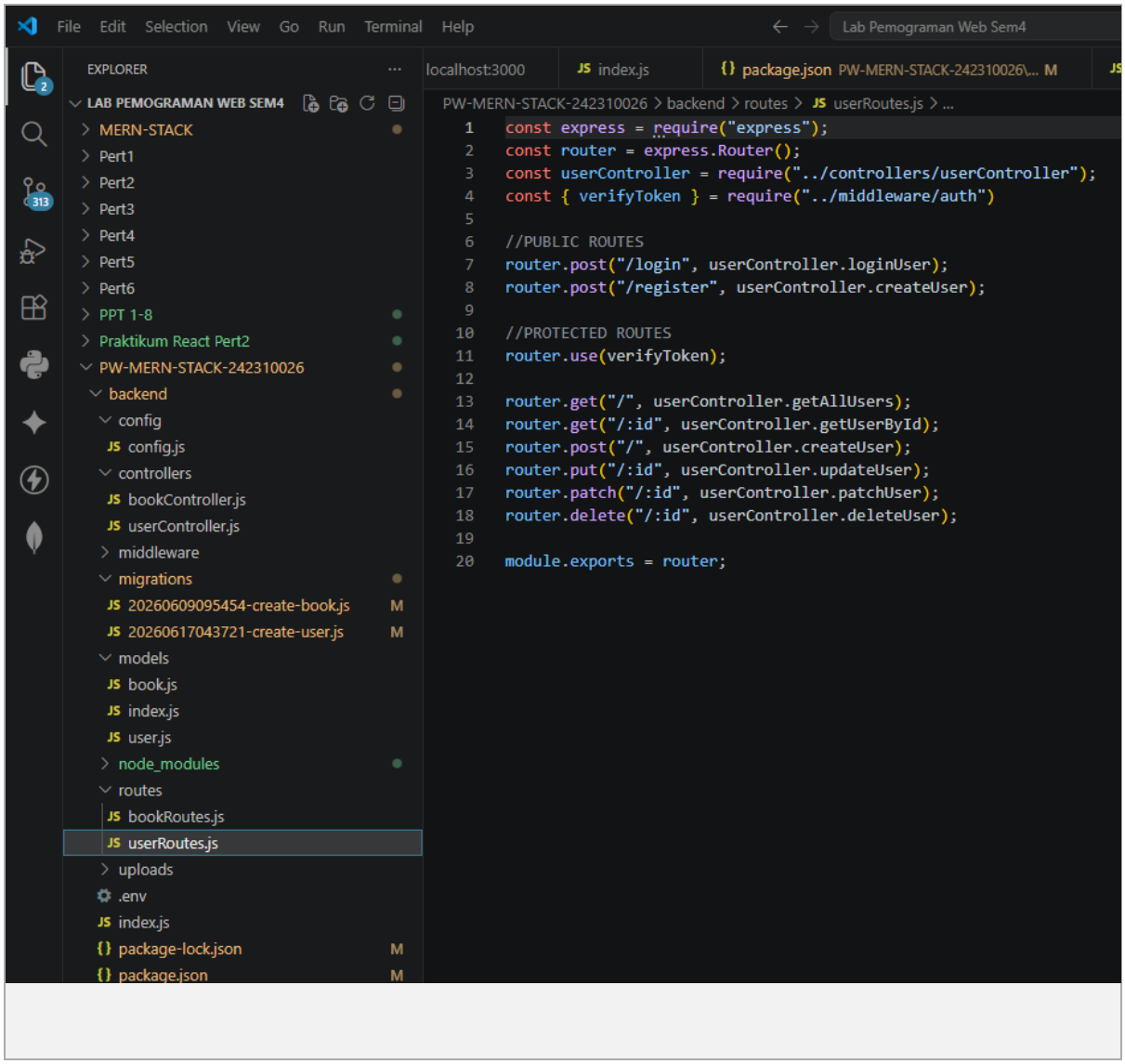

File Edit Selection View Go Run Terminal Help
Lab Pemograman Web Sem4
EXPLORER
LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
middleware
migrations
20260609095454-create-book.js M
20260617043721-create-user.js M
models
book.js
index.js
user.js
node_modules
routes
uploads
.env
index.js
package-lock.json M
package.json M
my-first-app-react
Quiz-2-Jam-Digital
node_modules
public
src
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
Quiz-2-Digital-Jam.docx
README.md
vite.config.js
Quiz-3-Web-Food
Quiz1
OUTLINE
TIMELINE
localhost:3000
index.js
package.json PW-MERN-STACK-242310026\... M
user.js
PW-MERN-STACK-242310026 > backend > migrations > 20260609095454-create-book.js > ...
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
async up(queryInterface, Sequelize) {
await queryInterface.createTable("books", {
id: {
allowNull: false,
autoIncrement: true,
primaryKey: true,
type: Sequelize.INTEGER,
},
title: {
type: Sequelize.STRING(255),
allowNull: false,
},
author: {
type: Sequelize.STRING(255),
allowNull: false,
},
rating: {
type: Sequelize.DECIMAL(3, 2),
allowNull: true,
defaultValue: 0.0,
},
views: {
type: Sequelize.INTEGER,
allowNull: true,
defaultValue: 0,
},
is_free: {
type: Sequelize.BOOLEAN,
allowNull: false,
defaultValue: false,
},
language: {
type: Sequelize.STRING(50),
allowNull: false,
defaultValue: "English",
},
sinopsis: {
type: Sequelize.TEXT,
allowNull: false,
},
story: {
type: Sequelize.TEXT("long"),
allowNull: false,
},
image: {
type: Sequelize.STRING(500),
allowNull: true,
},
created_by: {
type: Sequelize.STRING(255),
allowNull: true
}



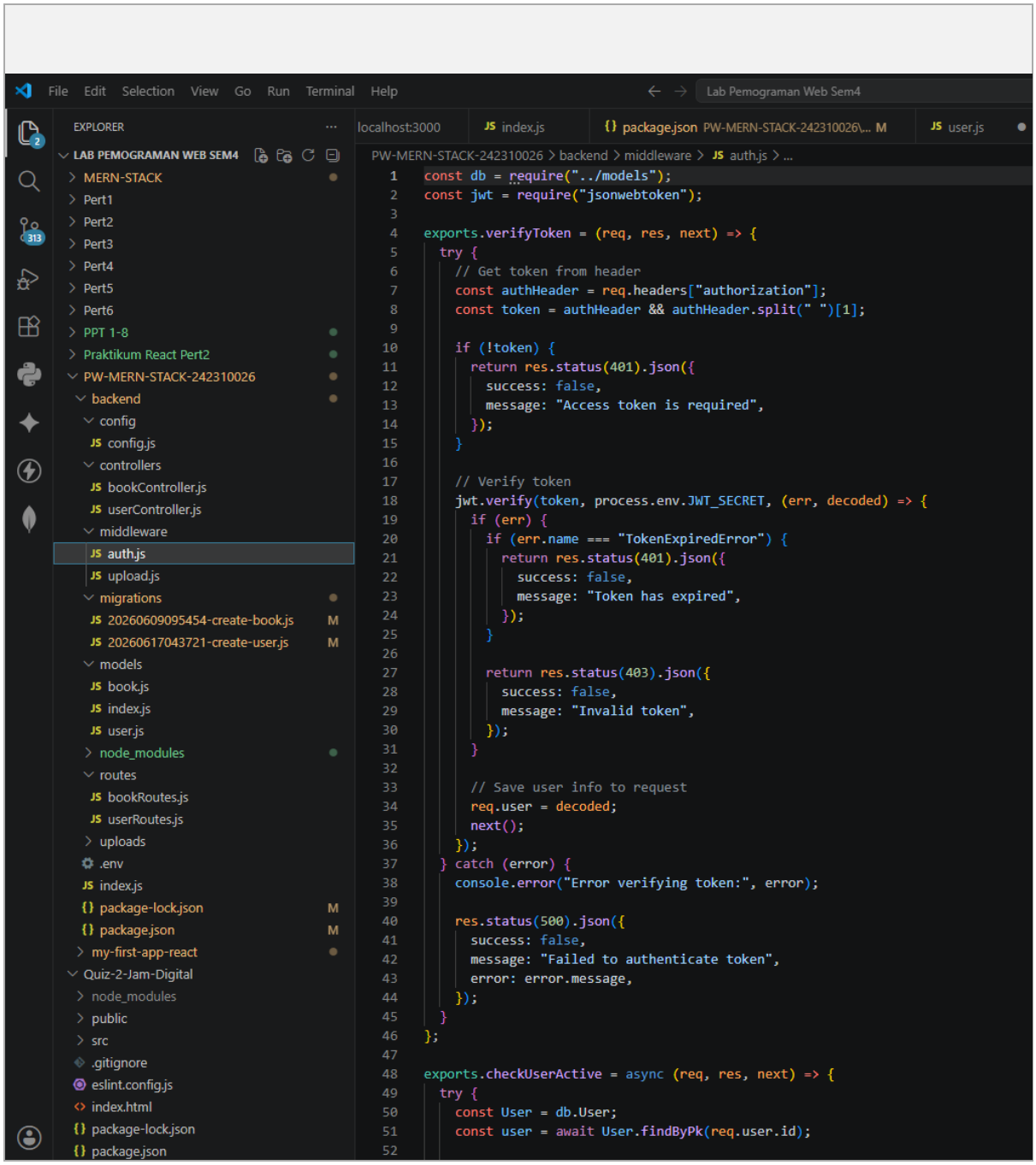


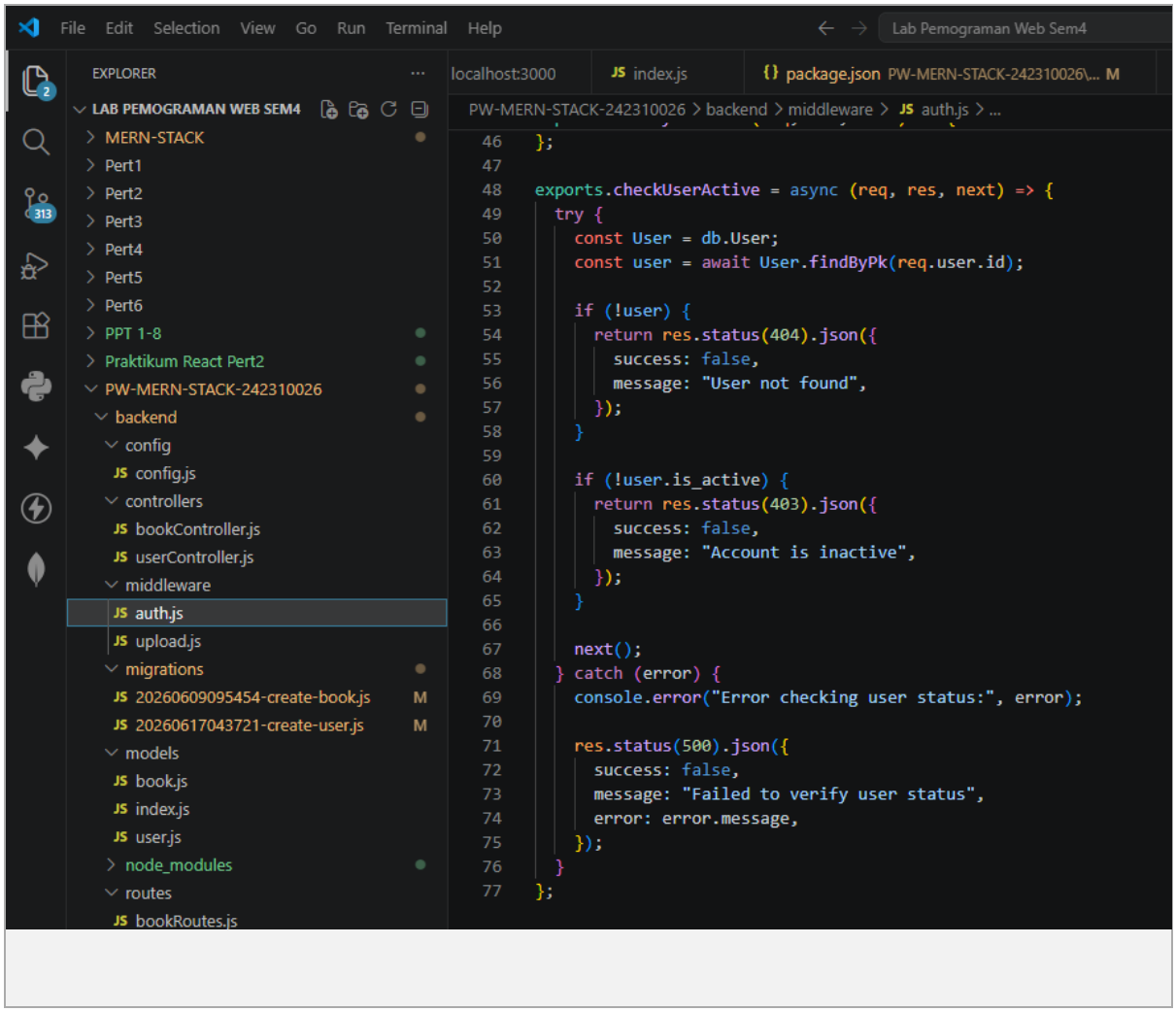
routes/userRoutes.js -> pemetaan endpoint /api/users

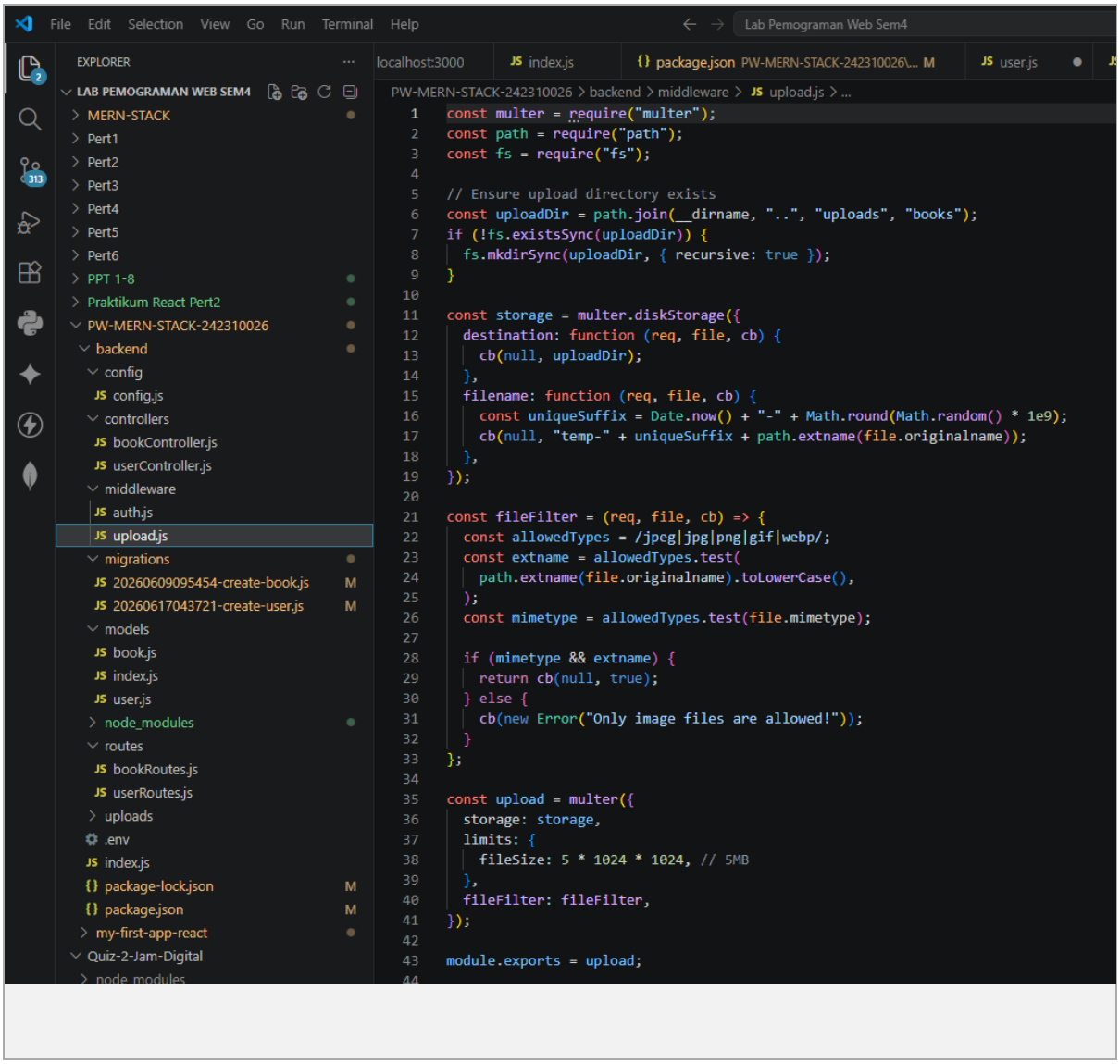

File Edit Selection View Go Run Terminal Help
Lab Pemograman Web Sem4
EXPLORER
LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
bookController.js
userController.js
middleware
migrations
20260609095454-create-book.js M
20260617043721-create-user.js M
models
book.js
index.js
user.js
node_modules
routes
bookRoutes.js
userRoutes.js
uploads
localhost:3000
index.js
package.json PW-MERN-STACK-242310026\... M
user.js
PW-MERN-STACK-242310026 > backend > routes > bookRoutes.js > ...
1 const express = require('express');
2 const router = express.Router();
3 const bookController = require("../controllers/bookController");
4 const upload = require("../middleware/upload");
5 const { verifyToken } = require("../middleware/auth")
6
7 //PROTECTED ROUTES
8 router.use(verifyToken);
9
10 //CRUD Routes
11 router.get("/", bookController.getAllBooks);
12 router.get("/:id", bookController.getBookById);
13 router.delete("/:id", bookController.deleteBook);
14
15 router.post("/", upload.single("coverImage"), bookController.createBook);
16 router.put("/:id", upload.single("coverImage"), bookController.updateBook);
17 router.patch("/:id", upload.single("coverImage"), bookController.patchBook);
18
19 module.exports = router;



middleware/auth.js -> verifikasi token JWT untuk protected routes


File Edit Selection View Go Run Terminal Help
Lab Pemograman Web Sem4
EXPLORER
LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
bookController.js
userController.js
middleware
auth.js
upload.js
migrations
20260609095454-create-book.js M
20260617043721-create-user.js M
models
book.js
index.js
user.js
node_modules
routes
bookRoutes.js
userRoutes.js
uploads
.env
index.js
package-lock.json M
package.json M
my-first-app-react
Quiz-2-Jam-Digital
node_modules
public
src
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
localhost:3000
index.js
package.json PW-MERN-STACK-242310026\... M
user.js
PW-MERN-STACK-242310026 > backend > middleware > auth.js > ...
1 const db = require("../models");
2 const jwt = require("jsonwebtoken");
3
4 exports.verifyToken = (req, res, next) => {
5 try {
6 // Get token from header
7 const authHeader = req.headers["authorization"];
8 const token = authHeader && authHeader.split(" ")[1];
9
10 if (!token) {
11 return res.status(401).json({
12 success: false,
13 message: "Access token is required",
14 });
15 }
16
17 // Verify token
18 jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
19 if (err) {
20 if (err.name === "TokenExpiredError") {
21 return res.status(401).json({
22 success: false,
23 message: "Token has expired",
24 });
25 }
26
27 return res.status(403).json({
28 success: false,
29 message: "Invalid token",
30 });
31 }
32
33 // Save user info to request
34 req.user = decoded;
35 next();
36 });
37 } catch (error) {
38 console.error("Error verifying token:", error);
39
40 res.status(500).json({
41 success: false,
42 message: "Failed to authenticate token",
43 error: error.message,
44 });
45 }
46 };
47
48 exports.checkUserActive = async (req, res, next) => {
49 try {
50 const User = db.User;
51 const user = await User.findByPk(req.user.id);
52


File Edit Selection View Go Run Terminal Help
Lab Pemograman Web Sem4
EXPLORER
LAB PEMOGRAMAN WEB SEM4
MERN-STACK
Pert1
Pert2
Pert3
Pert4
Pert5
Pert6
PPT 1-8
Praktikum React Pert2
PW-MERN-STACK-242310026
backend
config
config.js
controllers
bookController.js
userController.js
middleware
auth.js
upload.js
migrations
20260609095454-create-book.js M
20260617043721-create-user.js M
models
book.js
index.js
user.js
node_modules
routes
bookRoutes.js
localhost:3000
index.js
package.json PW-MERN-STACK-242310026\... M
PW-MERN-STACK-242310026 > backend > middleware > auth.js > ...
};
exports.checkUserActive = async (req, res, next) => {
try {
const User = db.User;
const user = await User.findByPk(req.user.id);
if (!user) {
return res.status(404).json({
success: false,
message: "User not found",
});
}
if (!user.is_active) {
return res.status(403).json({
success: false,
message: "Account is inactive",
});
}
next();
} catch (error) {
console.error("Error checking user status:", error);
res.status(500).json({
success: false,
message: "Failed to verify user status",
error: error.message,
});
}
};



Backend REST API untuk user management berhasil dibangun dan diuji menggunakan Postman, lengkap dengan autentikasi JWT dan hashing password menggunakan bcrypt. Ditemukan dan diperbaiki 2 bug terkait konsistensi penamaan tabel database dan konfigurasi script npm, sehingga project dapat dijalankan tanpa error.